

Module 1

Chapter 1 : Introduction (1.1 to 1.5)

Chapter 3 : Lexical Analysis (3.1 to 3.4)

1.1 Language Processors

- ▶ Compiler is a program that can read a program in one language the **Source language** and translate it into an equivalent program in another language the **Target language**
- ▶ An **important role of** a compiler is to **report any errors** in the source program that it detects during the translation.

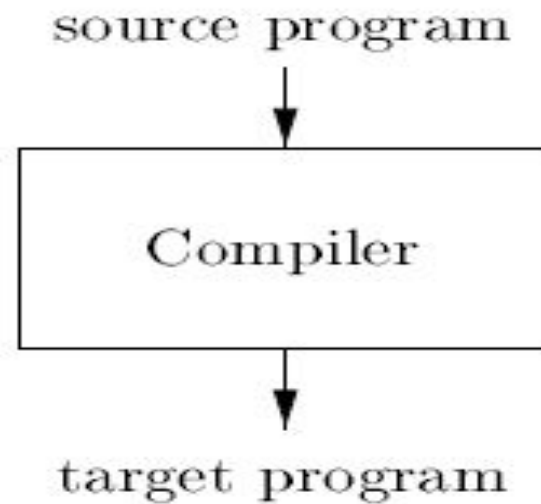


Figure 1.1: A compiler

Language Processors

- ▶ If the target program is an executable machine language program, it can be then called by the user to process inputs and produce outputs.

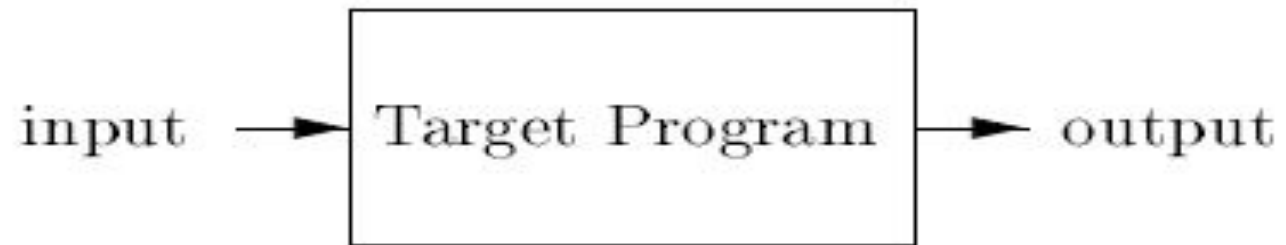


Figure 1.2: Running the target program

Language Processors

- ▶ An **interpreter** is another common kind of language processor. Instead of producing a target program as a translation, an interpreter appears to directly execute the operations specified in the source program on inputs supplied by the user.

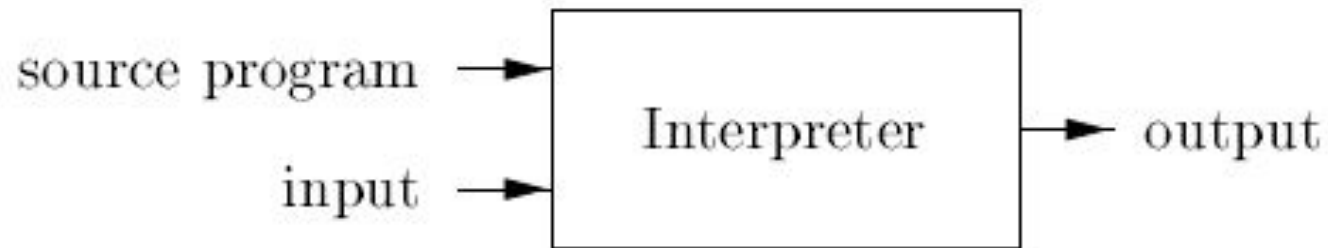


Figure 1.3: An interpreter

Language Processors

- ▶ A Java source program may first be compiled into an intermediate form called **bytecodes**.
- ▶ The bytecodes are then interpreted by a virtual machine
- ▶ A benefit of this arrangement is that bytecodes compiled on one machine can be interpreted on another machine .
- ▶ In order to achieve faster processing of inputs to outputs, some java compilers called **Just-in-time compilers**, translate the bytecodes into machine language immediately

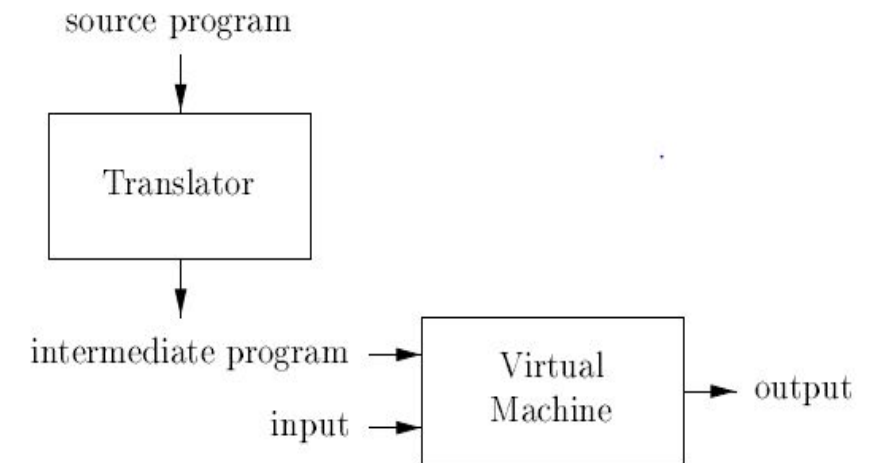


Figure 1.4: A hybrid compiler

Differences between Compilers and interpreters

Interpreter	Compiler
Translates program one statement at a time.	Scans the entire program and translates it as a whole into machine code.
It takes less amount of time to analyze the source code but the overall execution time is slower.	It takes large amount of time to analyze the source code but the overall execution time is comparatively faster.
No intermediate object code is generated, hence are memory efficient.	Generates intermediate object code which further requires linking, hence requires more memory.
Continues translating the program until the first error is met, in which case it stops. Hence debugging is easy.	It generates the error message only after scanning the whole program. Hence debugging is comparatively hard.
Programming language like Python, Ruby use interpreters.	Programming language like C, C++ use compilers.

Language Processing System

- ▶ In addition to a compiler, several other programs are required to create an executable target program.
- ▶ A source program may be divided into modules stored in separate files.
- ▶ The task of collecting the source program is sometimes entrusted to a separate program called pre-processor. The pre-processor may also expand shorthand called macros, into source language statements.
- ▶ The modified source program is then fed to compiler.
- ▶ The compiler may produce an assembly language program as its output, because the assembly language is easier to debug.

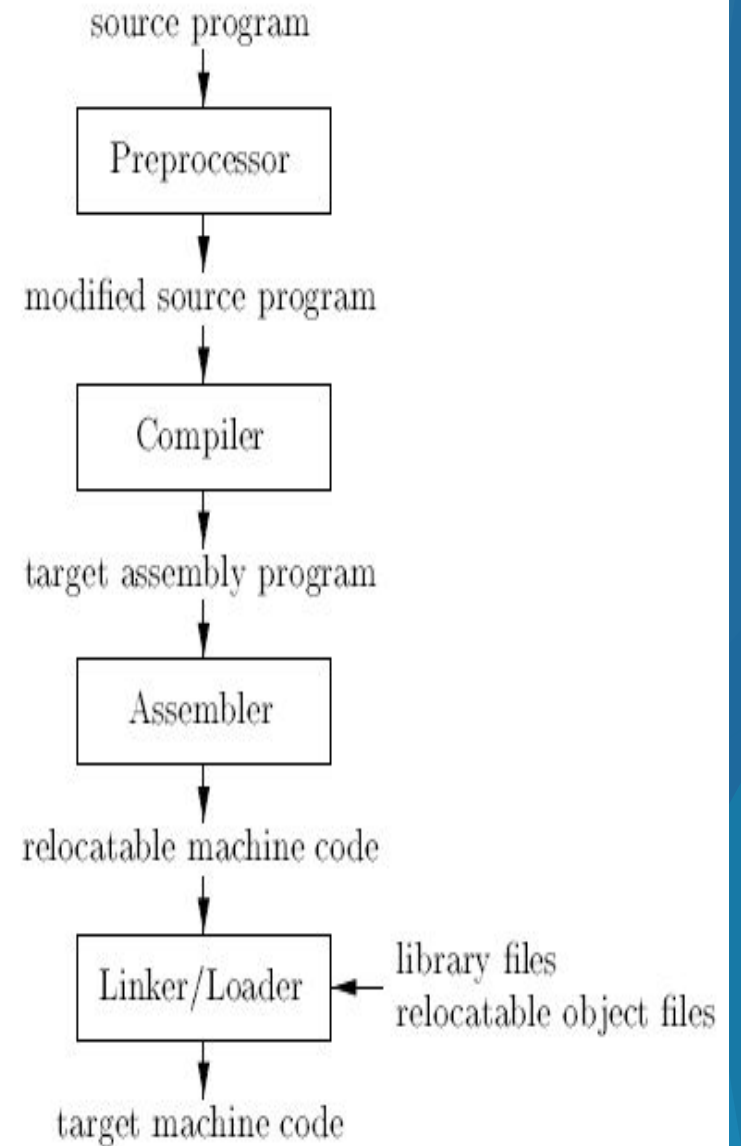


Figure 1.5: A language-processing system

Language Processing System

- ▶ The assembly language is then processed by a program called assembler
- ▶ Large programs are often compiled in pieces, so the relocatable machine code may have to be linked together with other relocatable object files and library files.
- ▶ The linker resolves **external memory addresses**, where the code in one file may refer to a location in another file.
- ▶ The loader then puts together all the executable object files into memory for execution.

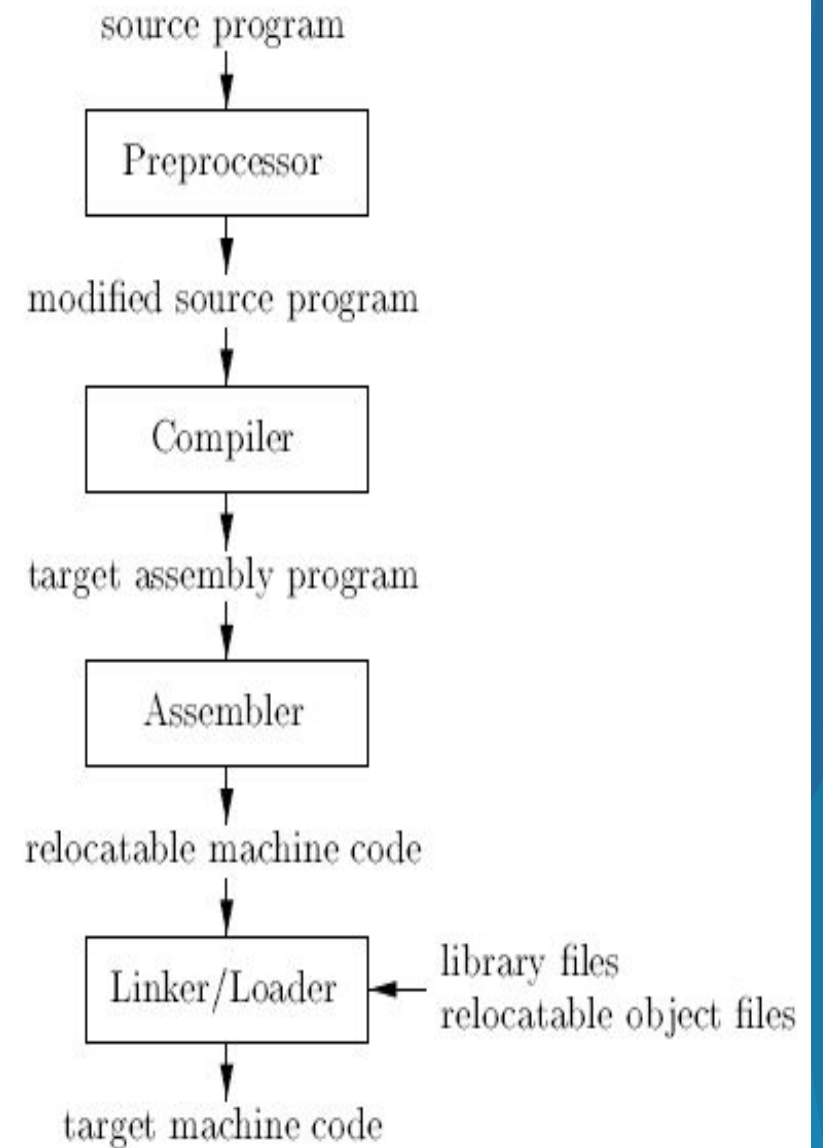
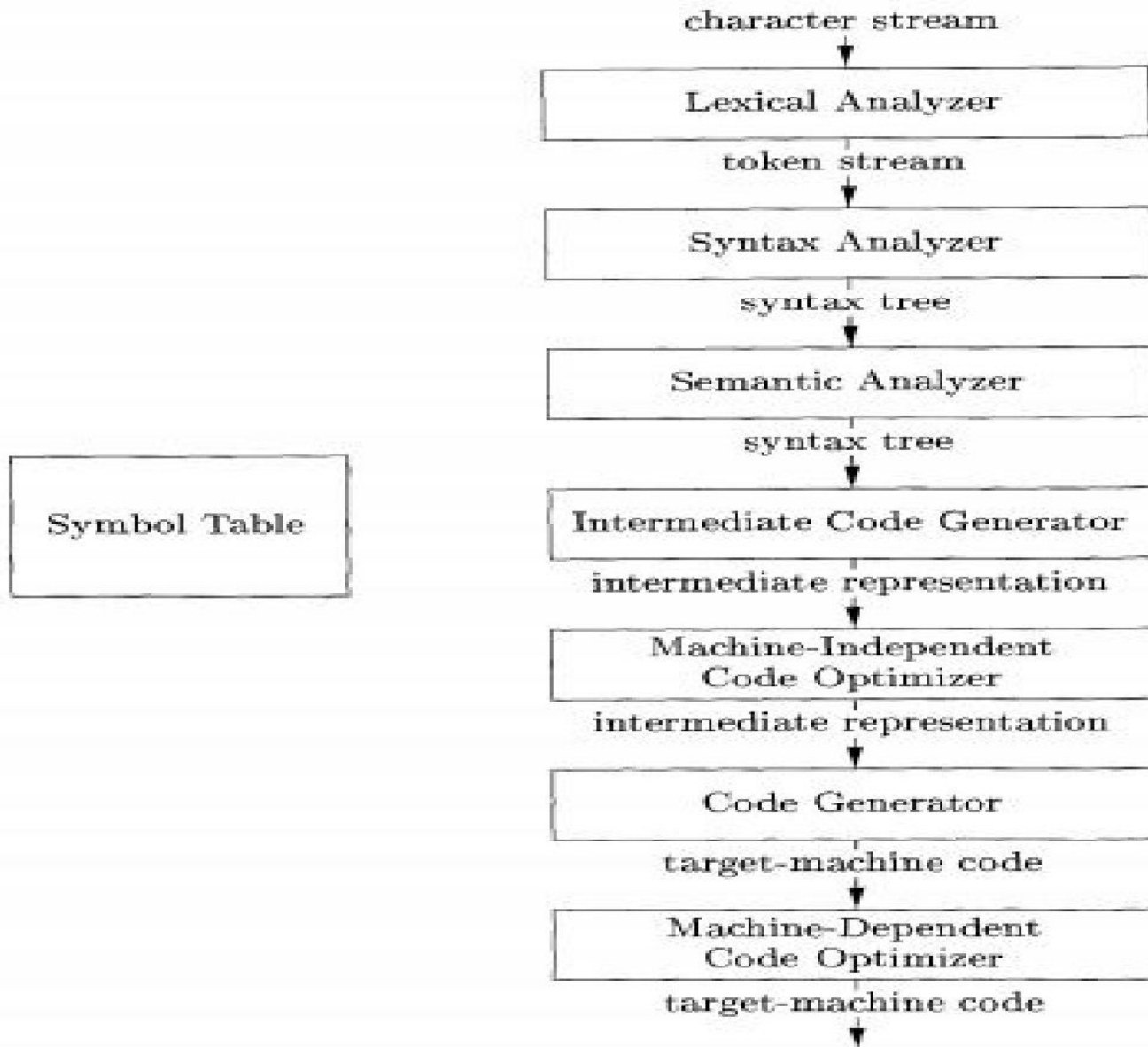


Figure 1.5: A language-processing system

1.2 Structure of Compilers



1.2 The structure of a Compiler

- ▶ The compiler consists of two parts:

Analysis and Synthesis

- ▶ Analysis

- ▶ The source program is read and broken down into constituents pieces and impose grammatical structure on them.
- ▶ It then uses this structure to create an intermediate code representation of the source program.
- ▶ The analysis part also collects information about the source program and stores it in a data structure called a **symbol table**, which is passed along with the intermediate representation to the synthesis part.
- ▶ Analysis is carried out in three phases: **Lexical Analysis, syntax analysis and semantic analysis.**
- ▶ Analysis is called the **front end of the compiler.**

The structure of a Compiler

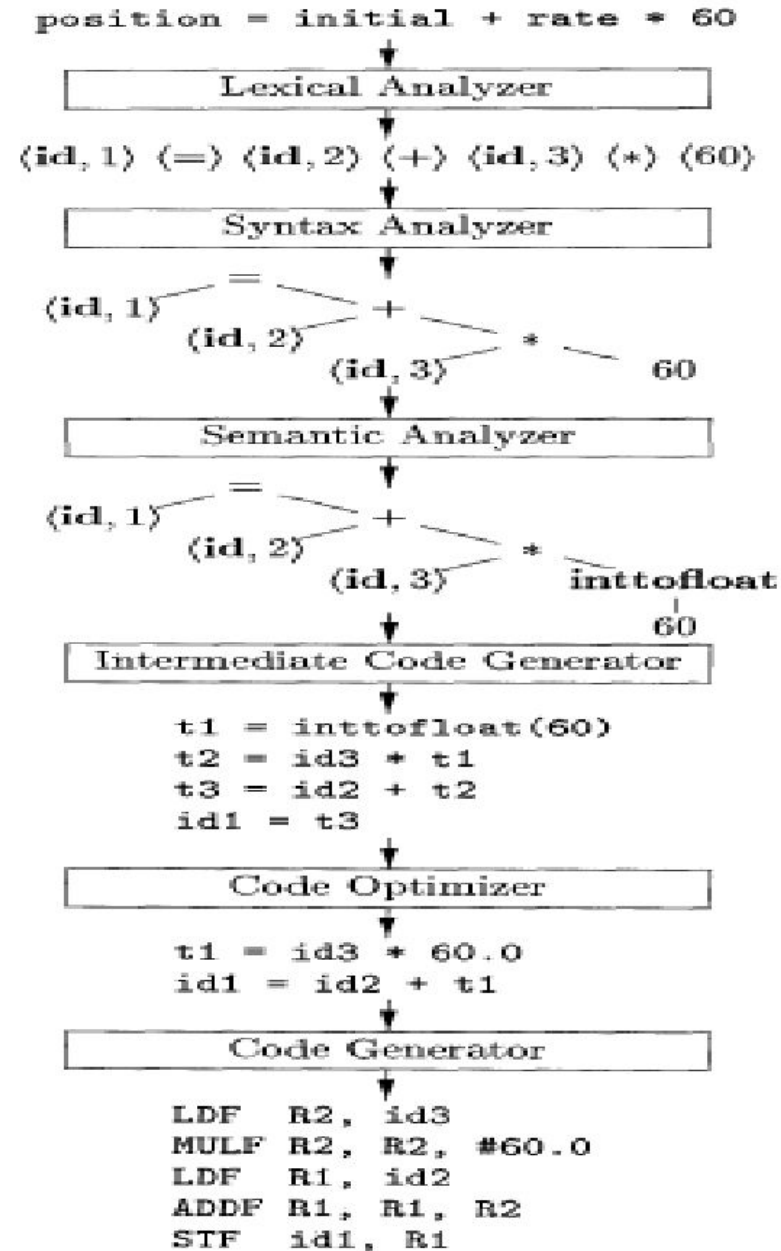
Synthesis

- ▶ In Synthesis part, the intermediate representation of the source language is taken and converted into an equivalent target program.
- ▶ During the process if certain code has to be optimized for efficient execution then the required code is optimized.
- ▶ Synthesis is carried out with the help of **intermediate code generation, code generation and code optimization**.
- ▶ Synthesis is called the **back end of the compiler**.

Translation of an assignment statement

1	position	...
2	initial	...
3	rate	...

SYMBOL TABLE



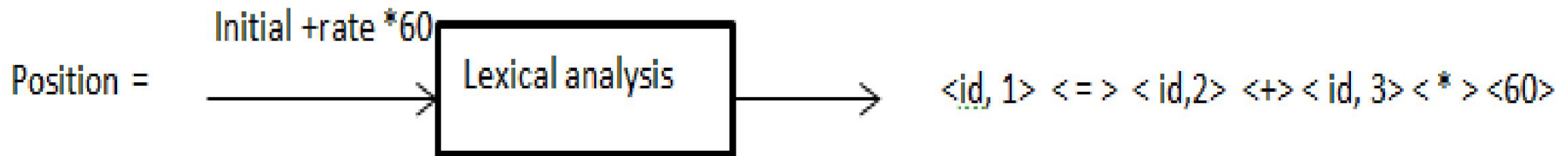
1. Lexical Analysis

- ▶ Lexical Analysis is also called **scanning**
- ▶ The lexical analyzer reads the stream of characters making up the source program and groups the characters into meaningful sequences called **lexemes**.
- ▶ For each lexeme, the lexical analyzer produces as output a token of the form.

<token name, attribute_value>

- ▶ Token name - is a symbol used in syntax analyzer
- ▶ Attribute_value- points to an entry in the symbol table, for this token.

Example: position = initial+rate *60

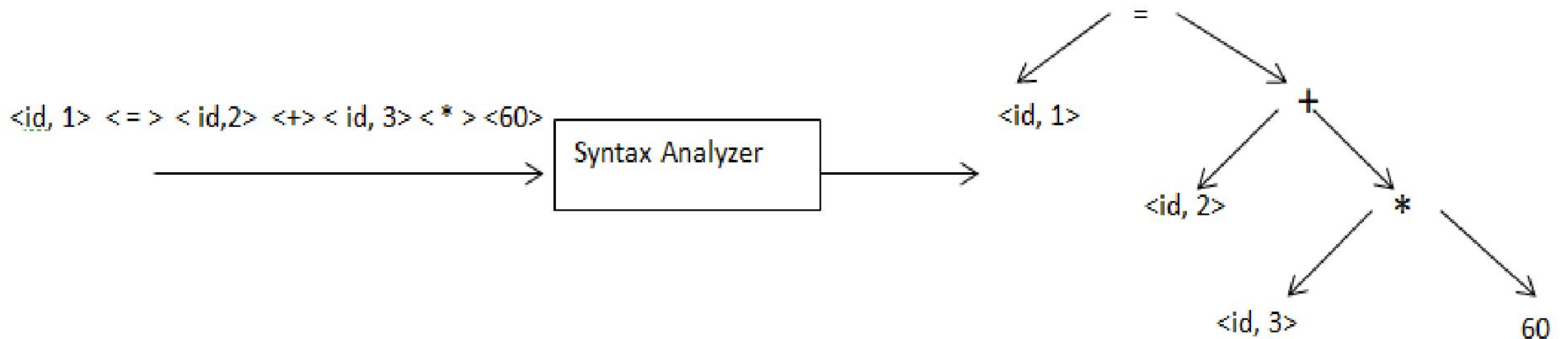


Lexical Analysis

- ▶ Where `id`, is an abstract symbol standing for identifier.
- ▶ 1,2,3 points to the symbol table entry.
- ▶ The symbol table entry for an identifier holds information about the identifier, such as its name and type.
- ▶ The blank characters which are used in the programming statement are eliminated during the lexical analysis phase.

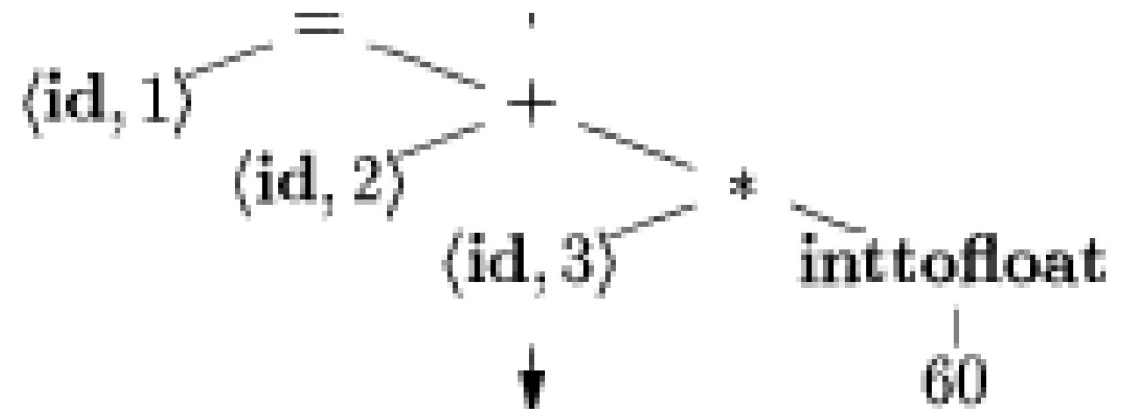
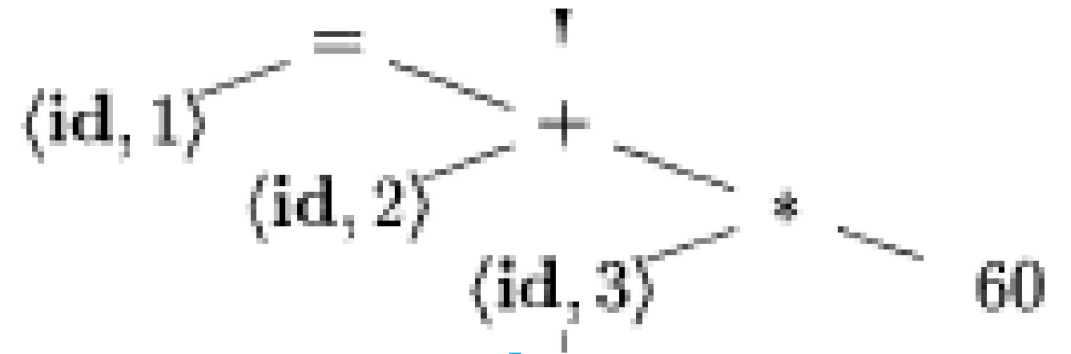
2. Syntax Analysis

- ▶ The syntax analysis is also called **parsing**.
- ▶ In this phase the tokens generated by the lexical analysis are grouped together to form a **hierarchical structure**.
- ▶ The hierarchical structure generated in their phase is called **parse tree** and **syntax tree**.
- ▶ **Syntax tree**: Each interior node represents an operation and children node represents the argument of the operation.



3. Semantic Analysis

- ▶ The semantic analyzer uses the syntax tree and the information in the symbol table to check the source program for semantic consistency with the language definition.
- ▶ It also gathers type information and saves it in either the syntax tree or the symbol table for subsequent use during the intermediate code generation.
- ▶ An important part of the semantic analysis is **type checking**



Semantic Analysis

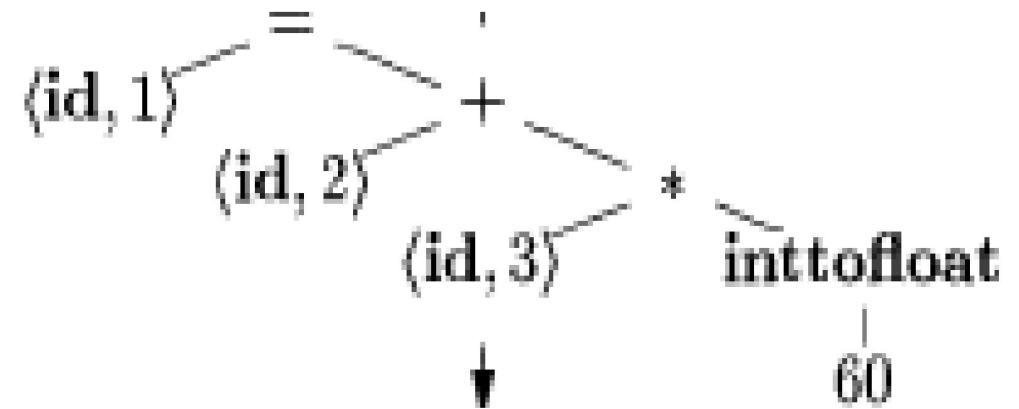
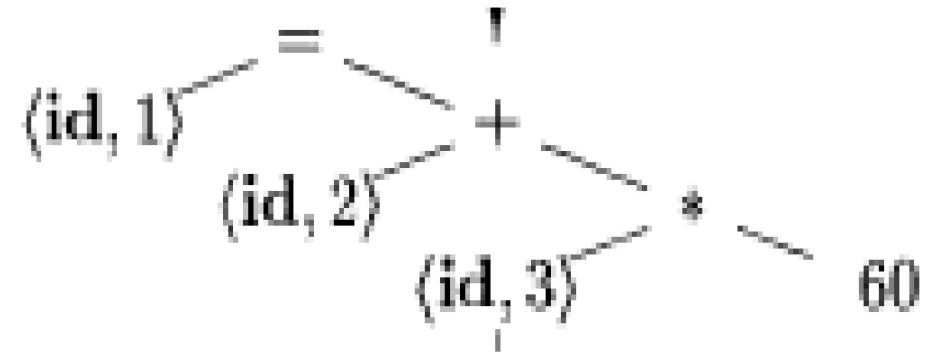
Semantic Errors:

Errors recognized by semantic analyzer are as follows:

- Type mismatch
- Undeclared variables
- Reserved identifier misuse

Functions of Semantic Analysis:

- Type Checking – Ensures that data types are used in a way consistent with their definition.
- Label Checking – A program should contain labels references.
- Flow Control Check – Keeps a check that control structures are used in a proper manner.(example: no break statement outside a loop)



4. Intermediate code generation

- ▶ Many compilers generate an explicit low level or machine like intermediate representation, which is a program for an abstract machine.
- ▶ Properties of Intermediate Code Generator- **should be easy to produce** and **easy to translate into target machine**.
- ▶ We consider an intermediate form called **three address code**, which consists of a sequence of assembly like instructions with three operands per instructions.
- ▶ Each operand can act like a register.
- ▶ Several points worth noting about three address code instructions:
 - ▶ Three address assignment has at most one operator on the right side.
 - ▶ Temporary name to hold the value computed.
 - ▶ Three address instruction may have fewer than three operands.

position = initial + rate *60

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

5. Code optimization

- ▶ The machine independent code optimization phase attempts to **improve the intermediate code**.
- ▶ This is necessary to have a faster executing code as **less consumption of memory**.
- ▶ Thus by optimizing the code the overall running time of the target program can be improved.
- ▶ The optimizer can deduce that the **conversion of 60 from integer to floating point can be done once** and for all at compile time, so the “**int to float**” operation can be eliminated by replacing the integer 60 by the floating-point number 60.0.
- ▶ t3 is used only once to transmit its value to id1 so the optimizer can transform into the shorter sequence.

```
t1 = id3 * 60.0  
id1 = id2 + t1
```

6. Code Generation

- ▶ The code generator takes as input an intermediate representation of the source program and maps it into the **target language**.
- ▶ The intermediate instructions are translated into sequences of machine instructions that perform the same task.
- ▶ For example, using registers R1 and R2, the intermediate code in (1 .4) might get translated into the machine code.

```
t1 = id3 * 60.0  
id1 = id2 + t1
```

```
LDF  R2,  id3  
MULF R2,  R2,  #60.0  
LDF  R1,  id2  
ADDF R1,  R1,  R2  
STF  id1,  R1
```

Code Generation

- ▶ The first operand of each instruction specifies a destination. The F in each instruction tells us that it deals with floating-point numbers.
- ▶ Loads the contents of address id3 into register R2, then multiplies it with floating-point constant 60.0.
- ▶ The # signifies that 60.0 is to be treated as an immediate constant.
- ▶ The third instruction moves id2 into register R1 and the fourth adds to it the value previously computed in register R2.
- ▶ Finally, the value in register R1 is stored into the address of id1, so the code correctly implements the assignment statement.

```
LDF  R2,  id3
MULF R2,  R2, #60.0
LDF  R1,  id2
ADDF R1,  R1, R2
STF  id1, R1
```

7. Symbol Table Management

- ▶ An essential function of a compiler is to record the variable names used in the source program and collect information about various attributes of each name.
- ▶ Attributes may provide information about the storage allocated for a **name**, its **type**, its **scope** , and in the case of **procedure names**, such things as the **number and types of its arguments**, the method of passing each argument , and the **type returned**.
- ▶ Symbol table is a data structure used to **store identifiers/variable names** used in the program.

8. Grouping of Phases into Passes

- ▶ Phases deals with the logical organization of a compiler.
- ▶ In an implementation, activities from several phases may be grouped together into a “pass” that reads an input file and writes an output file.
- ▶ For example,
 - ▶ the front-end phases of lexical analysis, syntax analysis, semantic analysis, and intermediate code generation might be grouped together into one pass.
 - ▶ Code optimization might be an optional pass.
 - ▶ Then there could be a back-end pass consisting of code generation for a particular target machine.
- ▶ In some compilers intermediate code generation might be in Back end pass.

9. Compiler Construction Tools

- ▶ The compiler writer, like any software developer, can profitably use modern software development environments containing tools such as language editors, debuggers, version managers, profilers, test harnesses, and so on.
- ▶ In addition to these, general software development tools, other more specialised tools have been created to implement various phases of compiler.
- ▶ The most successful tools are those that can be easily integrated into the remainder of the compiler. Thank you

Compiler Construction Tools

1. **Parser generators** that automatically produce syntax analyzers from a **grammatical description** of a programming language.
2. **Scanner generators** that produce lexical analysers from a **regular expression** description of the tokens of a language.
3. **Syntax-directed translation** engines that produce **collections of routines** for walking a parse tree and generating intermediate code.
4. **Code-generator** that produce a code generator from a **collection of rules** for translating each operation of the intermediate language into the machine language for a target machine.
5. **Data-flow analysis** engines that facilitate the gathering of information about **how values are transmitted** from one part of a program to each other part. Data-flow analysis is a key part of code optimization.
6. **Compiler-construction toolkits** that provide an **integrated set of routines** for constructing various phases of a compiler.

1.3 The Evolution of Programming Languages

- ▶ First electronic computers appeared – 1940
- ▶ Were programmed in Machine language by sequences of 0's and 1's that explicitly told the computer what operations to execute and in what order.
- ▶ The operations themselves were very low level:
 - ▶ move data from one location to another,
 - ▶ add the contents of two registers,
 - ▶ compare two values, and so on.
- ▶ This kind of programming was slow, tedious, and error prone. And once written, the programs were hard to understand and modify.

1. The Move to Higher-level Languages

- ▶ First step towards more people-friendly programming languages was the development of **mnemonic assembly languages** in the early 1950's.
- ▶ Initially - **assembly languages** and later - **Macro instructions**.
- ▶ After 1950's **Fortran** for scientific computation, **Cobol** for business data processing, and **LISP** for symbolic computation.
- ▶ In high level language programmers could more easily **write numerical computations, business applications, and symbolic programs**.
- ▶ Languages were so successful that they are still in use today.

The Move to Higher-level Languages

- ▶ Today, there are thousands of programming language.
 - ▶ First-generation languages - **machine languages**
 - ▶ Second-generation Languages - **assembly languages**
 - ▶ Third-generation - higher-level languages like **Fortran, Cobol, LISP, C, C++, C#, and Java**
 - ▶ Fourth-generation languages - languages designed for specific applications like **NOMAD** for report generation, **SQL** for database queries, and **Postscript** for text formatting
 - ▶ Fifth-generation language - has been applied to logic- and constraint-based languages like **Prolog and OPS5**.

The Move to Higher-level Languages

- ▶ Classification of languages uses the term
 - ▶ **Imperative** – languages in which a program specifies **how a computation is to be done**
Example: C, C++, C#,
 - ▶ **Declarative** for languages in which a program specifies **what computation is to be done**
Example: Prolog
- ▶ **Von Neumann language** is applied to programming languages whose computational model is based on the von Neumann computer architecture.
Example: C and Fortran
- ▶ An **object-oriented language** is one that supports object-oriented programming, a programming style in which a program consists of a collection of objects that interact with one another.
Example: C++, C#, Java and Ruby
- ▶ **Scripting languages** are interpreted languages with high-level operators designed for “gluing together” computations.
Example : JavaScript, Perl, PHP, Python, Ruby

2. Impact on Compilers

- ▶ Advances in programming languages placed new demands on compiler writers.
- ▶ They had to devise algorithms and representations to translate and support the new language features.
- ▶ Not only did the **compiler writers** have to track **new language features**, they also had to **devise translation algorithms** that would take maximal advantage of the **new hardware capabilities**.
- ▶ Compilers can help promote the use of high-level languages by minimizing the execution overhead of the programs written in these languages
- ▶ Performance of a computer system is so dependent on compiler technology that compilers are used as a tool in evaluating architectural concepts before a computer is built.
- ▶ Compiler writing is challenging. A compiler by itself is a large program.
- ▶ A compiler must translate correctly the potentially infinite set of programs that could be written in the source language

1.4 The Science of Building a Compiler

- ▶ Compiler design is full of examples where complicated real-world problems are solved by abstracting the essence of the problem mathematically.
- ▶ Example:
 - ▶ take a problem,
 - ▶ formulate a **mathematical abstraction** that captures the key characteristics, and
 - ▶ solve it using **mathematical techniques**.
- ▶ A compiler must accept all source programs that conform to the specification of the language;
 - ▶ the set of source programs is **infinite**
 - ▶ any program can be very large consisting of possibly millions of lines of code
- ▶ Any transformation performed by the compiler while translating a source program must preserve the meaning of the program being compiled.

1. Modelling in Compiler Design and Implementation

- ▶ The study of compilers is mainly a study of
 - ▶ how we design the **right mathematical models**
 - ▶ choose the **right algorithms**,
 - ▶ balancing the need for generality and power against simplicity and efficiency.
- ▶ Most fundamental models are **finite-state machines and regular expressions**. These models are useful for
 - ▶ describing the **lexical units of programs** (keywords, identifiers, and such)
 - ▶ For describing the algorithms used by the compiler to recognize those units
- ▶ Most fundamental models are **context-free grammars** used to
 - ▶ describe the **syntactic structure** of programming languages such as the nesting of parentheses or control constructs.
- ▶ **Trees** are an important model for representing the structure of programs and their translation into object code

2. The Science of Code Optimization

- ▶ Optimization - produce code that is more efficient than the obvious code.
- ▶ the optimization of code that a compiler performs has become both **more important** and **more complex**
 - ▶ It is more complex because **processor architectures** have become more complex, yielding more opportunities to improve the way code executes
 - ▶ It is more important because **massively parallel computers** require substantial optimization, or their performance suffers by orders of magnitude
- ▶ **multicore machines** - all compilers will have to face the problem of taking advantage of multiprocessor machines.
- ▶ The use of a **rigorous mathematical foundation** allows us to show that an optimization is correct and that it produces the desirable effect for all possible inputs.

The Science of Code Optimization

- ▶ Compiler optimizations must meet the following design objectives:
 - ▶ The optimization **must be correct**, that is, preserve the meaning of the compiled program
 - ▶ The optimization must **improve the performance** of many programs
 - ▶ The compilation time must be **kept reasonable**, and
 - ▶ The engineering effort required must be **manageable**

1.5 Applications of Compiler Technology (Self Learning Topic)

- ▶ Implementation of High-Level Programming Languages
- ▶ Optimizations for Computer Architectures
- ▶ Design of New Computer Architectures
- ▶ Program Translations
- ▶ Software Productivity Tools

1. Implementation of High-Level Programming Languages

- ▶ high-level programming language defines a programming abstraction:
 - ▶ The programmer expresses an algorithm using the language,
 - ▶ the compiler must translate that program to the target language
- ▶ higher-level programming languages are easier to program in, but are less efficient, that is, the target programs run more slowly.
- ▶ Programmers using a low-level language have more control over a computation and can, in principle, produce more efficient code
- ▶ lower-level programs are harder to write and worse still less portable, more prone to errors, and harder to maintain

Implementation of High-Level Programming Languages

- ▶ Popular choice of programming languages have been in the direction of increased levels of abstraction
- ▶ C - 1980
- ▶ 90's C++; Java
- ▶ all common programming languages, including C, Fortran and Cobol,
 - ▶ support user-defined aggregate data types, such as arrays and structures,
 - ▶ and high-level control flow, such as loops and procedure invocations.

Implementation of High-Level Programming Languages

- ▶ Object orientation was first introduced in Simula in 1967, and has been incorporated in languages such as Smalltalk, C++, C#, and Java.
- ▶ The key ideas behind object orientation are
 - ▶ Data abstraction and
 - ▶ Inheritance of properties
- ▶ Both of which have been found to make programs more modular and easier to maintain
- ▶ Object-oriented programs are different from those written in many other languages, in that they consist of many more, but smaller procedures

2. Optimizations for Computer Architectures

- ▶ The rapid evolution of computer architectures has also led to a demand for new compiler technology.
- ▶ all high-performance systems take advantage of the same two basic techniques: parallelism and memory hierarchies.
- ▶ Parallelism can be found at several levels:
 - ▶ at the instruction level - where multiple operations are executed simultaneously
 - ▶ at the processor level - where different threads of the same application are run on different processors.
- ▶ Memory hierarchies are a response to the basic limitation that we can build very fast storage or very large storage, but not storage that is both fast and large

Parallelism

- ▶ All modern microprocessors exploit instruction-level parallelism.
- ▶ This parallelism can be hidden from the programmer.
- ▶ Programs are written as if all instructions were executed in sequence; the hardware dynamically checks for dependencies in the sequential instruction stream and issues them in parallel when possible
- ▶ In some cases, the machine includes a hardware scheduler that can change the instruction ordering to increase the parallelism in the program.
- ▶ Whether the hardware reorders the instructions or not, compilers can rearrange the instructions to make instruction-level parallelism more effective.

Memory Hierarchies

- ▶ memory hierarchy consists of several levels of storage with different
 - ▶ speeds
 - ▶ sizes (with the level closest to the processor being the fastest but smallest)
- ▶ The average memory-access time of a program is reduced if most of its accesses are satisfied by the faster levels of the hierarchy
- ▶ Both parallelism and the existence of a memory hierarchy improve the potential performance of a machine
- ▶ Memory hierarchies are found in all machines
 - ▶ small number of registers consisting of hundreds of bytes
 - ▶ caches containing kilobytes to megabytes
 - ▶ physical memory containing megabytes to gigabytes,
 - ▶ secondary storage that contains gigabytes and beyond
- ▶ the speed of accesses between adjacent levels of the hierarchy can differ by two or three orders of magnitude.
- ▶ The performance of a system is often limited not by the speed of the processor but by the performance of the memory subsystem.

3. Design of New Computer Architectures

- ▶ In the early days of computer architecture design, compilers were developed after the machines were built
- ▶ That has changed. Since programming in high-level languages is the norm, the performance of a computer system is determined not by its raw speed but also by how well compilers can exploit its features
- ▶ In modern computer architecture development, compilers are developed in the processor-design stage and compile the code in the simulator to determine the feature

Design of New Computer Architectures - RISC and CISC

- ▶ One of the best known examples of how compilers influenced the design of computer architecture was the invention of the RISC (Reduced Instruction-Set Computer) architecture.
- ▶ Prior to this invention, the trend was to develop progressively complex instruction sets intended to make assembly programming easier; these architectures were known as CISC (Complex Instruction-Set Computer)
- ▶ CISC instruction sets include
- ▶ complex memory-addressing modes
 - ▶ to support data-structure accesses
 - ▶ procedure-invocation instructions that save registers and pass parameters on the stack
- ▶ Most general-purpose processor architectures, including PowerPC, SPARC, MIPS, Alpha, and PA-RISC, are based on the RISC concept

Design of New Computer Architectures - Specialized Architectures

- ▶ Over the last three decades, many architectural concepts have been proposed
 - ▶ data flow machines,
 - ▶ vector machines,
 - ▶ VLIW (Very Long Instruction Word) machines,
 - ▶ SIMD (Single Instruction, Multiple Data)
 - ▶ arrays of processors,
 - ▶ systolic arrays,
 - ▶ multiprocessors with shared memory, and
 - ▶ multiprocessors with distributed memory.

4. Program Translations

- ▶ While we normally think of compiling as a translation from a high-level language to the machine level, the same technology can be applied to translate between different kinds of languages
- ▶ The following are some of the important applications of program-translation techniques.
 - ▶ Binary Translation
 - ▶ Hardware Synthesis
 - ▶ Database Query Interpreters
 - ▶ Compiled Simulation
- ▶ Compiler technology can be used to translate the binary code for one machine to that of another, allowing a machine to run programs originally compiled for another instruction set

Program Translations

- ▶ Not only is most software written in high-level languages; even hardware designs are mostly described in high-level hardware description languages like Verilog and VHDL
- ▶ Hardware designs are typically described at the register transfer level (RTL), where variables represent registers and expressions represent combinational logic
- ▶ Hardware-synthesis tools translate RTL descriptions automatically into gates, which are then mapped to transistors and eventually to a physical layout.
- ▶ Besides specifying software and hardware, languages are useful in many other applications, For example, query languages, especially SQL are used to search databases.
- ▶ Database queries consist of predicates containing relational and Boolean operators.

Program Translations

- ▶ Simulation is a general technique used in many scientific and engineering disciplines to understand a phenomenon or to validate a design - expensive
- ▶ Inputs to a simulator usually include the description of the design and specific input parameters for that particular simulation run.
- ▶ Compiled simulation is used in many state-of-the-art tools that simulate designs written in Verilog or VHDL.

5. Software Productivity Tools

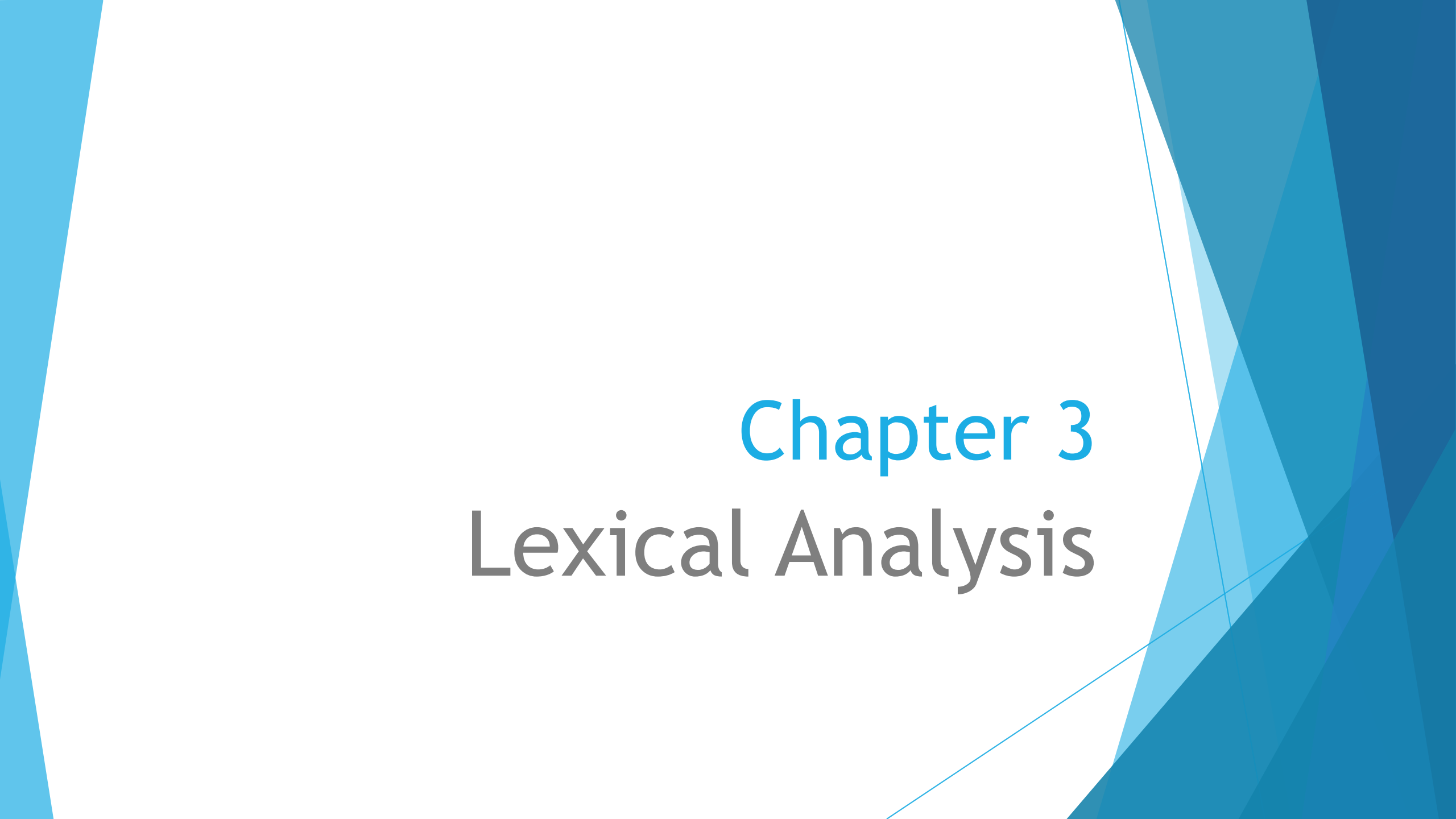
- ▶ Programs are arguably the most complicated engineering artifacts ever produced; they consist of many details, every one of which must be correct before the program will work completely
- ▶ As a result, errors are rampant in programs;
 - ▶ errors may crash a system, produce wrong results,
 - ▶ render a system vulnerable to security attacks,
 - ▶ or even lead to catastrophic failures in critical systems.
- ▶ Testing is the primary technique for locating errors in programs
- ▶ complementary approach is to use data-flow analysis to locate errors statically, Data-flow analysis can find errors along all the possible execution paths,

Type Checking and Bound checking

- ▶ **Type checking** is an effective and well-established technique to catch inconsistencies in programs.
- ▶ It can be used to catch errors, for example, where an operation is applied to the wrong type of object, or if parameters passed to a procedure do not match the signature of the procedure.
- ▶ **Bounds Checking** - It is easier to make mistakes when programming in a lower-level language than a higher-level one.
- ▶ For example, in systems are caused by buffer over flows in programs written in C. Because C does not have array-bounds checks, it is up to the user to ensure that the arrays are not accessed out of bounds.

Memory-Management Tools

- ▶ Garbage collection is another excellent example of the trade-off between efficiency and a combination of ease of programming and software reliability.
- ▶ Automatic memory management obliterates all memory-management errors which are a major source of problems in C and C++ programs
- ▶ Various tools have been developed to help programmers find memory management errors
- ▶ Example: Purify – dynamically catches memory management errors.

The background features abstract, overlapping geometric shapes in various shades of blue, ranging from light sky blue to deep navy blue. These shapes are primarily located on the left and right sides of the frame, creating a modern, angular aesthetic. The central area is a plain white background where the text is located.

Chapter 3

Lexical Analysis

The Role of the Lexical Analyzer

- ▶ **Definition:** The process of reading the source program and converting it into sequence of tokens is called **lexical analysis**.
- ▶ The **main task** of the lexical analyser is to **read the input characters of the source program, group them into lexemes**, and **produce as output a sequence of tokens** for each lexeme in the source program
- ▶ The **stream of tokens** is sent to the **parser for syntax analysis**
- ▶ It is common for the **lexical analyzer** to interact with the **symbol table** as well.
- ▶ When the lexical analyzer discovers a lexeme constituting an identifier, it needs to enter that lexeme into the symbol table.
- ▶ In some cases, information regarding the kind of identifier may be read from the symbol table by the lexical analyzer to assist it in determining the proper token it must pass to the parser.

The Role of the Lexical Analyzer

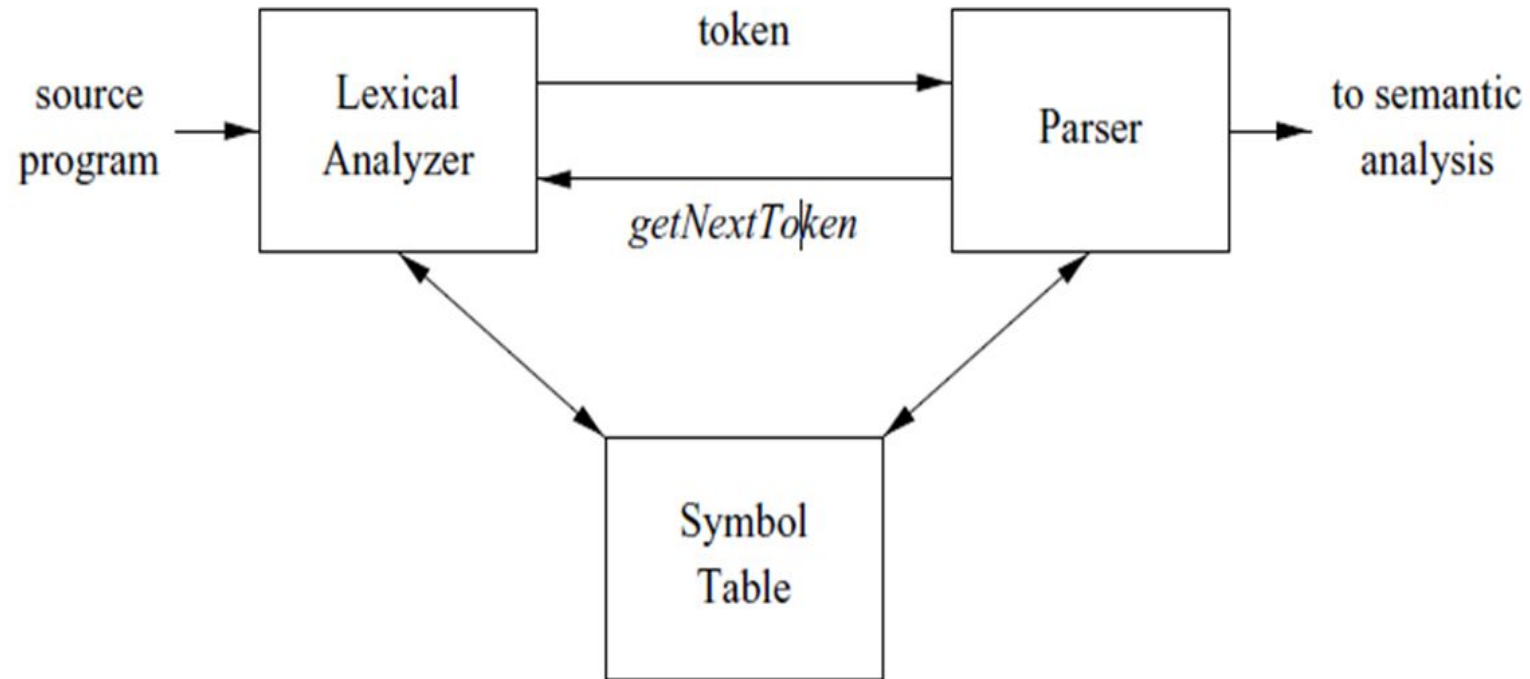


Figure 3.1: Interactions between the lexical analyzer and the parser

The Role of the Lexical Analyzer

Other Tasks Performed by lexical analyzer

- ▶ Stripping out comments and whitespace. (blank, newline, tab, and perhaps other characters that are used to separate tokens in the input)
- ▶ Correlating error messages generated by the compiler with the source program
- ▶ For instance, the lexical analyser may keep track of the number of newline characters seen, so it can associate a line number with each error message.
- ▶ In some compilers, the lexical analyser makes a copy of the source program with the error messages inserted at the appropriate positions.

The Role of the Lexical Analyzer

Other Tasks Performed by lexical analyzer

- ▶ Sometimes, lexical analysers are divided into a cascade of two processes:
 - ▶ **Scanning** consists of the simple processes that do not require tokenization of the input, such as deletion of comments and compaction of consecutive whitespace characters into one
 - ▶ **Lexical analysis** is the more complex portion, which produces tokens from the output of the scanner

The Role of the Lexical Analyzer

Lexical Analysis Versus Parsing

- ▶ ***Simplicity*** of design is the most important consideration. The separation of lexical and syntactic analysis often allows us to simplify at least one of these tasks.
- ▶ ***Compiler efficiency*** is improved- separate lexical analyser-serve only the lexical task, and also specialized buffering techniques-reading input characters can speed up.
- ▶ ***Compiler portability*** is enhanced-Input-device-specific peculiarities can be restricted to the lexical analyser

The Role of the Lexical Analyzer

Tokens, Patterns, and Lexemes

► *Token*

- A token is a pair consisting of a token name and an optional attribute value.
- The token name is an abstract symbol representing a kind of lexical unit.
- Ex : identifier, keyword (if, while, ...), punctuation character, multi-character operator (:=, ->, ...).

► *Patterns*

- A pattern is a description of the form that the lexemes of a token may take.
- The pattern is just the sequence of characters that form the keyword
- For identifiers and some other tokens, the pattern is a more complex structure that is matched by many strings.
- Ex : letter followed by letters and digits is a pattern that matches a word like x or y with the token id (= identifier).

The Role of the Lexical Analyzer

Tokens, Patterns, and Lexemes

► *Lexeme*

- A lexeme is a sequence of characters in the source program that matches the pattern for a token and is identified by the lexical analyser as an instance of that token.
- Example : `printf ( "Total = %d\n" , score ) ;`
- both **printf** and **score** are lexemes matching the pattern for token `id`, and `" Total = %d\n"` is a lexeme matching literal.
- the following classes cover most or all of the tokens:

TOKEN	INFORMAL DESCRIPTION	SAMPLE LEXEMES
if	characters <code>i</code> , <code>f</code>	<code>if</code>
else	characters <code>e</code> , <code>l</code> , <code>s</code> , <code>e</code>	<code>else</code>
comparison	<code><</code> or <code>></code> or <code><=</code> or <code>>=</code> or <code>==</code> or <code>!=</code>	<code><=</code> , <code>!=</code>
id	letter followed by letters and digits	<code>pi</code> , <code>score</code> , <code>D2</code>
number	any numeric constant	<code>3.14159</code> , <code>0</code> , <code>6.02e23</code>
literal	anything but <code>"</code> , surrounded by <code>"</code> 's	<code>"core dumped"</code>

Figure 3.2: Examples of tokens

The Role of the Lexical Analyzer

Attributes of Tokens

- ▶ In many cases the lexical analyser returns to the parser not only a token name, but an attribute value that describes the lexeme represented by the token ;
- ▶ The token name influences parsing decisions, while the attribute value influences translation of tokens after the parse.

Example 3.2: The token names and associated attribute values for the Fortran statement

`E = M * C ** 2`

are written below as a sequence of pairs.

`<id, pointer to symbol-table entry for E>`
`<assign_op>`
`<id, pointer to symbol-table entry for M>`
`<mult_op>`
`<id, pointer to symbol-table entry for C>`
`<exp_op>`
`<number, integer value 2>`

The Role of the Lexical Analyzer

Lexical Errors

- ▶ It is hard for a lexical analyser to tell , without the aid of other components, that there is a source-code error.
- ▶ For instance, if the string `fi` is encountered for the first time in a C program in the context

```
fi ( a == f(x)) ...
```

- ▶ probably the parser in this case - handle an error due to transposition of the letters
- ▶ The simplest recovery strategy is "panic mode" recovery.
- ▶ We delete successive characters from the remaining input, until the lexical analyzer can find a well-formed token at the beginning of what input is left

The Role of the Lexical Analyzer

Lexical Errors

- ▶ Other possible error-recovery actions are:
 - ▶ 1. Delete one character from the remaining input.
 - ▶ 2. Insert a missing character into the remaining input.
 - ▶ 3. Replace a character by another character.
 - ▶ 4. Transpose two adjacent characters.

Input Buffering

Buffer Pairs

- ▶ Because of large amount of time consumption in moving characters, specialized buffering techniques have been developed to reduce the amount of overhead required to process an input character
- ▶ Fig shows the buffer pairs which are used to hold the input data.

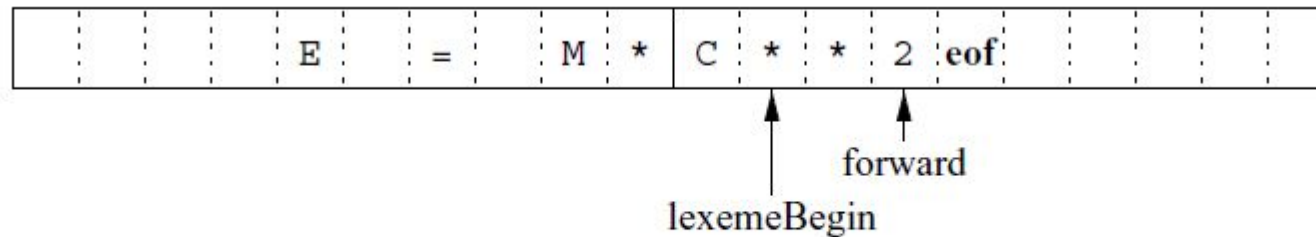


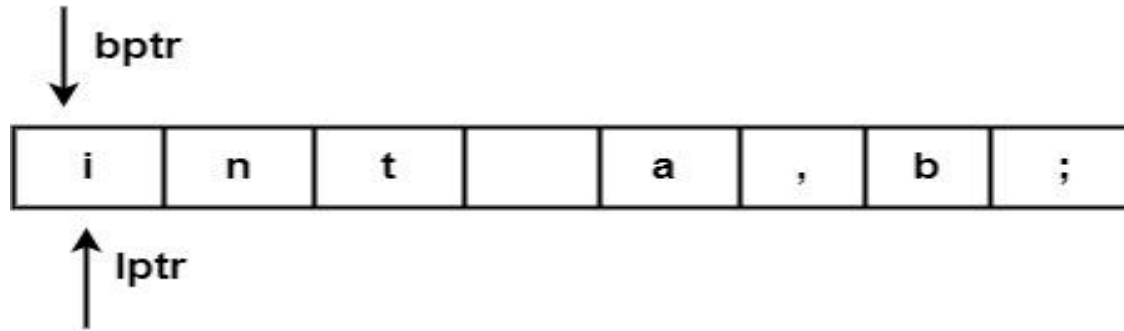
Figure 3.3: Using a pair of input buffers

Input Buffering

▶ Scheme

- ▶ Consists of two buffers, each consists of N-character size which are reloaded alternatively.
- ▶ N-Number of characters on one disk block, e.g., 4096.
- ▶ N characters are read from the input file to the buffer using one system read command.
- ▶ eof is inserted at the end if the number of characters is less than N.

Input Buffering

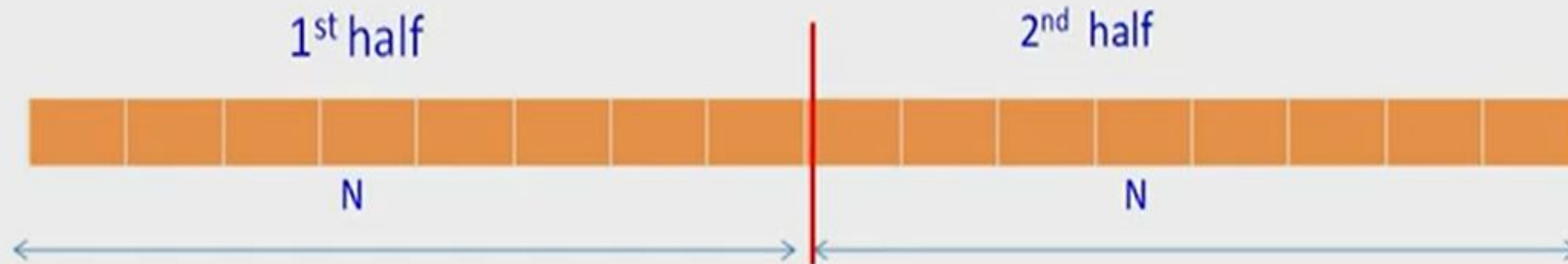


► Pointers

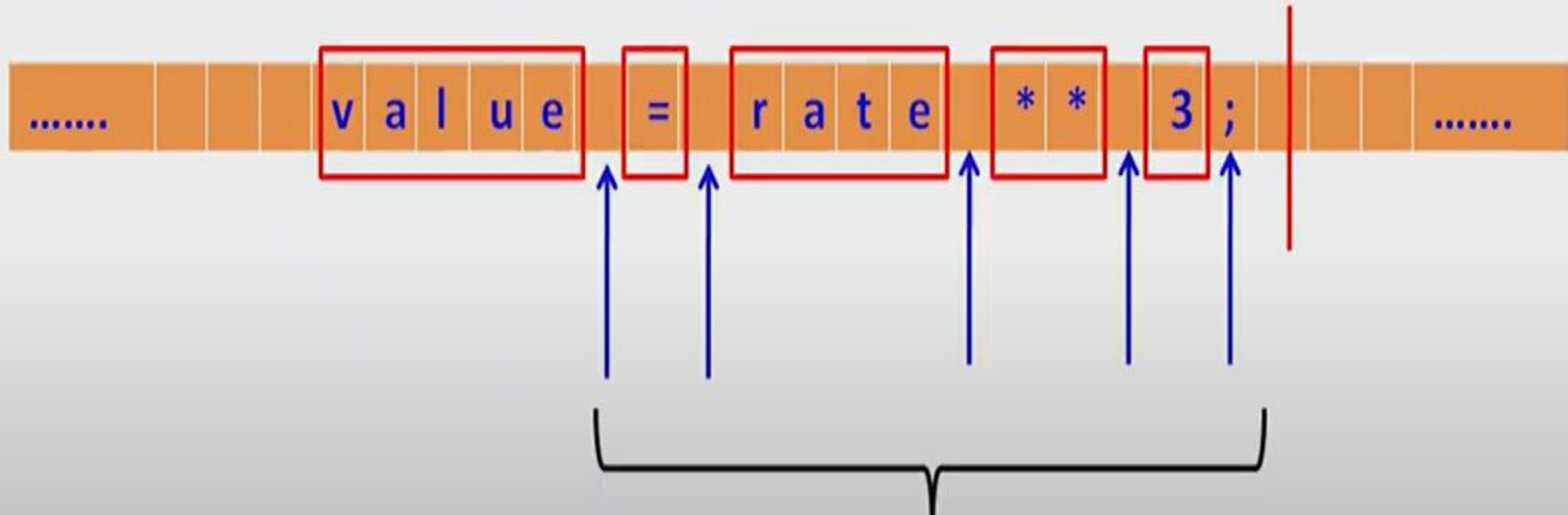
- Two pointers lexemeBegin and forward are maintained.
- lexeme Begin points to the beginning of the current lexeme which is yet to be found or attempting to determine.
- forward scans ahead until a match for a pattern is found.
- Once a lexeme is found, lexemeBegin is set to the character immediately after the lexeme which is just found and forward is set to the character at its right end.
- Current lexeme is the set of characters between two pointers.

Buffer Pairs

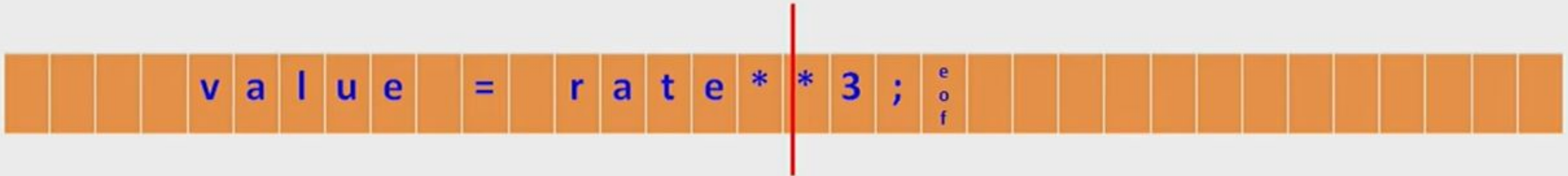
- Buffer is divided into **two N-characters halves**
- N is number of characters
 - eg: 1024 or 4096.



```
value = rate ** 3;
```



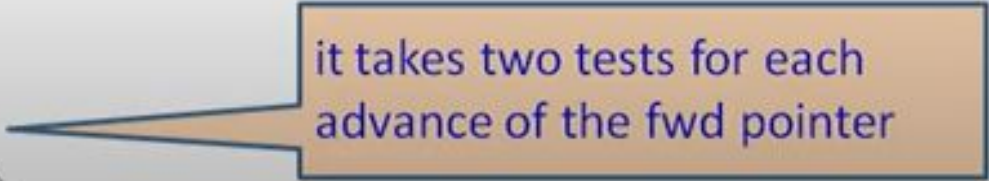
Buffer Pairs



- Read N bytes into **one half** of the buffer each time.
- If input has less than N bytes, put a special **EOF** marker in the buffer
- When one buffer has been processed, read N bytes into the **other buffer** half

Program code

```
if (fwd at end of first half) ←  
    reload second half;  
    set fwd to point to beginning of second half;  
else if (fwd at end of second half) ←  
    reload first half;  
    set fwd to point to beginning of first half;  
else  
    fwd++;
```



it takes two tests for each
advance of the fwd pointer

Input Buffering

Sentinels

- ▶ In the previous scheme, each time when the forward pointer is moved, a check is done to ensure that one half of the buffer has not moved off. If it is done, then the other half must be reloaded.
- ▶ Therefore, the end of the buffer halves require two tests for each advance of the forward pointer.
- ▶ We can combine the buffer-end test with the test for the current character if we extend each buffer to hold a sentinel character at the end.
- ▶ The sentinel is a special character that cannot be part of the source program, and a natural choice is the character eof .

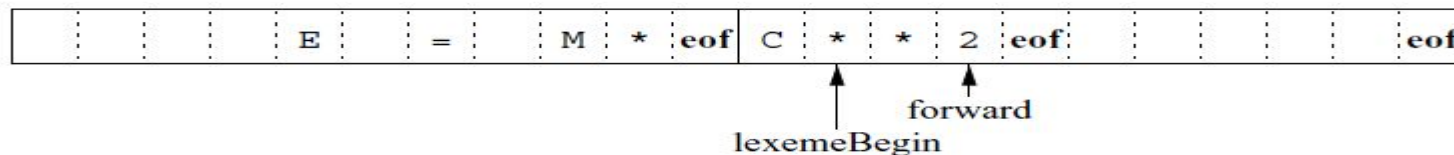
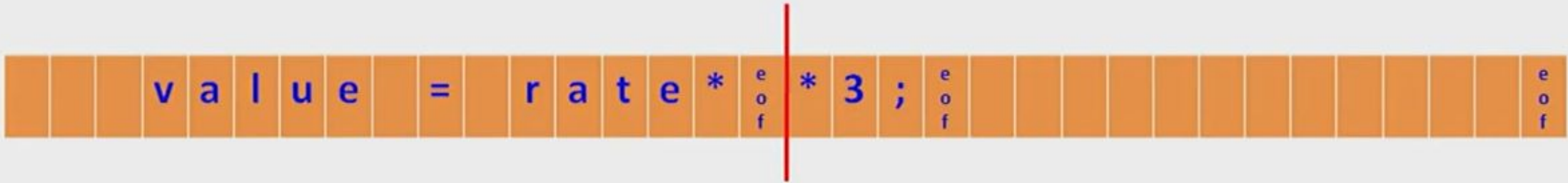


Figure 3.4: Sentinels at the end of each buffer

Buffer Pairs with sentinels



- Adds extra *eof* character at the end as sentinel
- Extend each buffer half to hold a *sentinel* at the end
- It optimizes the code by reducing the number of tests to one per advance of fwd

Input Buffering

Algorithm

```
switch ( *forward++ ) {  
    case eof:  
        if (forward is at end of first buffer ) {  
            reload second buffer;  
            forward = beginning of second buffer;  
        }  
        else if (forward is at end of second buffer ) {  
            reload first buffer;  
            forward = beginning of first buffer;  
        }  
        else /* eof within a buffer marks the end of input */  
            terminate lexical analysis;  
        break;  
    Cases for the other characters  
}
```

Figure 3.5: Lookahead code with sentinels

Specification of Tokens

1. Strings and Languages
2. Operation on languages
3. Regular Expressions
4. Regular Definition

Strings and Languages

- ▶ An **alphabet** is any finite set of symbols
 - ▶ Example : symbols are letters, digits, and punctuation.
 - ▶ The set $\{0,1\}$ is the binary alphabet
 - ▶ ASCII is an important example of an alphabet;

- ▶ A **string** over an alphabet is a finite sequence of symbols drawn from that alphabet
- ▶ Terms “sentence” and “word” are often used as synonyms for “string.”
- ▶ The length of a string s , usually written $|s|$, is the number of occurrences of symbols in s
 - ▶ Example; $s = \text{flower}$ and $|s| = 6$, empty string = ϵ $|s| = 0$

- ▶ A **language** is any countable set of strings over some fixed alphabet.
- ▶ If x and y are strings, then the concatenation of x and y , denoted xy , is the string formed by appending y to x .

Terms for Parts of Strings

1. A *prefix* of string s is any string obtained by removing zero or more symbols from the end of s . For example, `ban`, `banana`, and ϵ are prefixes of `banana`.
2. A *suffix* of string s is any string obtained by removing zero or more symbols from the beginning of s . For example, `nana`, `banana`, and ϵ are suffixes of `banana`.
3. A *substring* of s is obtained by deleting any prefix and any suffix from s . For instance, `banana`, `nan`, and ϵ are substrings of `banana`.
4. The *proper* prefixes, suffixes, and substrings of a string s are those, prefixes, suffixes, and substrings, respectively, of s that are not ϵ or not equal to s itself.
5. A *subsequence* of s is any string formed by deleting zero or more not necessarily consecutive positions of s . For example, `baan` is a subsequence of `banana`.

Operation on languages

OPERATION	DEFINITION AND NOTATION
<i>Union of L and M</i>	$L \cup M = \{s \mid s \text{ is in } L \text{ or } s \text{ is in } M\}$
<i>Concatenation of L and M</i>	$LM = \{st \mid s \text{ is in } L \text{ and } t \text{ is in } M\}$
<i>Kleene closure of L</i>	$L^* = \bigcup_{i=0}^{\infty} L^i$
<i>Positive closure of L</i>	$L^+ = \bigcup_{i=1}^{\infty} L^i$

Figure 3.6: Definitions of operations on languages

Operation on languages

1. $L \cup D$ is the set of letters and digits — strictly speaking the language with 62 strings of length one, each of which strings is either one letter or one digit.
2. LD is the set of 520 strings of length two, each consisting of one letter followed by one digit.
3. L^4 is the set of all 4-letter strings.
4. L^* is the set of all strings of letters, including ϵ , the empty string.
5. $L(L \cup D)^*$ is the set of all strings of letters and digits beginning with a letter.
6. D^+ is the set of all strings of one or more digits.

Regular Expressions

- ▶ Regular expressions are mathematical symbolisms which describe the set of strings of a specific language.
- ▶ It provides a convenient and useful notation for representing tokens.

Example: Regular expression for an Identifier in C language.

- ▶ `letter_(letter_ | digit)*`
- ▶ The vertical bar above means union, the parentheses are used to group subexpressions, the star means zero or more occurrences

Regular Expressions

- ▶ The regular expressions are built recursively out of smaller regular expressions, by following rules
- ▶ Each regular expression r denotes a language $L(r)$, which is also defined recursively from the languages denoted by r 's subexpressions.
- ▶ BASIS: There are rules that form the basis:
 - ▶ 1. $\emptyset$ is a regular expression over empty set $\{ \}$.
 - ▶ 2. ϵ is a regular expression, and $L(\epsilon)$ is $\{\epsilon\}$, that is, the language whose sole member is the empty string.
 - ▶ 3. If a is a symbol in Σ , then a is a regular expression, and $L(a) = \{a\}$, that is, the language with one string, of length one, with a in its one position.

Regular Expressions

INDUCTION: There are four parts to the induction whereby larger regular expressions are built from smaller ones. Suppose r and s are regular expressions denoting languages $L(r)$ and $L(s)$, respectively.

1. $(r)|(s)$ is a regular expression denoting the language $L(r) \cup L(s)$.
2. $(r)(s)$ is a regular expression denoting the language $L(r)L(s)$.
3. $(r)^*$ is a regular expression denoting $(L(r))^*$.
4. (r) is a regular expression denoting $L(r)$. This last rule says that we can add additional pairs of parentheses around expressions without changing the language they denote.

Regular Expressions

- ▶ As defined, regular expressions often contain unnecessary pairs of parentheses. We may drop certain pairs of parentheses if we adopt the conventions that:
 - ▶ a) The unary operator $*$ has highest precedence and is left associative
 - ▶ b) Concatenation has second highest precedence and is left associative
 - ▶ c) $|$ has lowest precedence and is left associative.
- ▶ for example, we may replace the regular expression

$$(a) | ((b)^*(c)) \quad \Rightarrow \quad a | b^*c$$

Regular Expressions

1. The regular expression $\mathbf{a|b}$ denotes the language $\{a, b\}$.
2. $\mathbf{(a|b)(a|b)}$ denotes $\{aa, ab, ba, bb\}$, the language of all strings of length two over the alphabet Σ . Another regular expression for the same language is $\mathbf{aa|ab|ba|bb}$.
3. $\mathbf{a^*}$ denotes the language consisting of all strings of zero or more a 's, that is, $\{\epsilon, a, aa, aaa, \dots\}$.
4. $\mathbf{(a|b)^*}$ denotes the set of all strings consisting of zero or more instances of a or b , that is, all strings of a 's and b 's: $\{\epsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$. Another regular expression for the same language is $\mathbf{(a^*b^*)^*}$.
5. $\mathbf{a|a^*b}$ denotes the language $\{a, b, ab, aab, aaab, \dots\}$, that is, the string a and all strings consisting of zero or more a 's and ending in b .

Regular Expressions

LAW	DESCRIPTION
$r s = s r$	$ $ is commutative
$r (s t) = (r s) t$	$ $ is associative
$r(st) = (rs)t$	Concatenation is associative
$r(s t) = rs rt; (s t)r = sr tr$	Concatenation distributes over $ $
$\epsilon r = r\epsilon = r$	ϵ is the identity for concatenation
$r^* = (r \epsilon)^*$	ϵ is guaranteed in a closure
$r^{**} = r^*$	$*$ is idempotent

Figure 3.7: Algebraic laws for regular expressions

Regular Definitions

- ▶ For notational convenience, we may wish to give names to certain regular expressions and use those names in subsequent expressions, as if the names were themselves symbols.
- ▶ If Σ is an alphabet of basic symbols, then a regular definition is a sequence of definitions of the form:

$d_1 \rightarrow r_1$

$d_2 \rightarrow r_2$

....

$d_n \rightarrow r_n$

- ▶ Where: 1. Each d_i is a new symbol, not in Σ and not the same as any other of the d 's, and
2. Each r_i is a regular expression over the alphabet $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$

Regular Definitions

Example 3.7: Using these shorthands, we can rewrite the regular definition of Example 3.5 as:

$$\begin{aligned} \textit{letter_} &\rightarrow [\textit{A-Za-z_}] \\ \textit{digit} &\rightarrow [0-9] \\ \textit{id} &\rightarrow \textit{letter_} (\textit{letter_} \mid \textit{digit})^* \end{aligned}$$

The regular definition of Example 3.6 can also be simplified:

$$\begin{aligned} \textit{digit} &\rightarrow [0-9] \\ \textit{digits} &\rightarrow \textit{digit}^+ \\ \textit{number} &\rightarrow \textit{digits} (. \textit{digits})? (\textit{E} [+-]? \textit{digits})? \end{aligned}$$

Extensions of Regular Expressions

1. *One or more instances.* The unary, postfix operator $^+$ represents the positive closure of a regular expression and its language.
2. *Zero or one instance.* The unary postfix operator $^?$ means “zero or one occurrence.” That is, $r^?$ is equivalent to $r|\epsilon$, or put another way, $L(r^?) = L(r) \cup \{\epsilon\}$. The $^?$ operator has the same precedence and associativity as $*$ and $^+$.
3. *Character classes.* A regular expression $a_1|a_2|\cdots|a_n$, where the a_i 's are each symbols of the alphabet, can be replaced by the shorthand $[a_1a_2\cdots a_n]$.

Recognition of Tokens

- ▶ Token is usually represented by a pair token type and token value.
- ▶ The token type tells us the category of the token and the token value gives us the information regarding token.
- ▶ The token value is also called as token attribute.

LEXEMES	TOKEN NAME	ATTRIBUTE VALUE
Any <i>ws</i>	—	—
if	if	—
then	then	—
else	else	—
Any <i>id</i>	id	Pointer to table entry
Any <i>number</i>	number	Pointer to table entry
<	relop	LT
<=	relop	LE
=	relop	EQ
<>	relop	NE
>	relop	GT
>=	relop	GE

Recognition of Tokens

<i>digit</i>	→	[0-9]
<i>digits</i>	→	<i>digit</i> ⁺
<i>letter</i>	→	[A-Za-z]
<i>id</i>	→	<i>letter</i> (<i>letter</i> <i>digit</i>)*
<i>if</i>	→	if
<i>then</i>	→	then
<i>else</i>	→	else
<i>relop</i>	→	< > <= >= = <>

Figure 3.11: Patterns for tokens of Example 3.8



Transition Diagram

Transition Diagrams

- Lexical analysis uses transition diagram to keep track of information about characters that are seen as the forward pointer scans the input
 - Transition diagrams are also called finite automata
-
- Positions in a transition diagram are drawn as circles and are called states.
 - The states are connected by arrows, called edges.
 - A double circle indicated an accepting state, a state in which a token is found.
 - a^* indicates that input retraction must take place

Recognition of Tokens

Transition Diagram

- ▶ Some important conventions about transition diagrams are:
 1. Certain states are said to be **accepting**, or final. These states indicate that a lexeme has been found and indicating an accepting state by a double circle.
 - ▶ if there is an action to be taken | typically returning a token and an attribute value to the parser. we shall attach that action to the accepting state
 2. In addition, if it is necessary to retract the forward pointer one position (i.e., the lexeme does not include the symbol that got us to the accepting state), then we shall additionally place a * near that accepting state.
 - ▶ In our example, it is never necessary to retract forward by more than one position, but if it were, we could attach any number of *'s to the accepting state.
 3. One state is designated the start state, or initial state; it is indicated by an edge, labelled “start,” entering from nowhere. The transition diagram always begins in the start state before any input symbols have been read/used.

Recognition of Tokens

Transition Diagram

LEXEMES	TOKEN NAME	ATTRIBUTE VALUE
Any <i>ws</i>	–	–
if	if	–
then	then	–
else	else	–
Any <i>id</i>	id	Pointer to table entry
Any <i>number</i>	number	Pointer to table entry
<	relop	LT
<=	relop	LE
=	relop	EQ
<>	relop	NE
>	relop	GT
>=	relop	GE

Figure 3.12: Tokens, their patterns, and attribute values

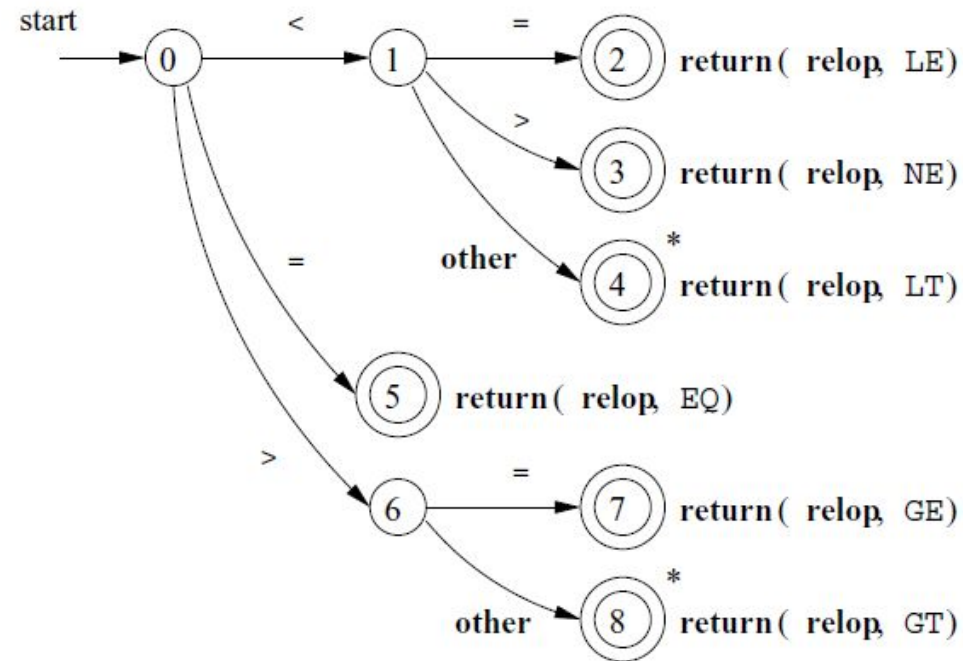


Figure 3.13: Transition diagram for **relop**

Recognition of Tokens

Recognition of Reserved Words and Id

Transition Diagram for Identifiers and Keywords



`gettoken( )`: returns token (`id`, `if`, `then`,...) if it looks
the symbol table

`install_id( )`: return 0 if keyword or a pointer to the
symbol table entry if `id`

Example

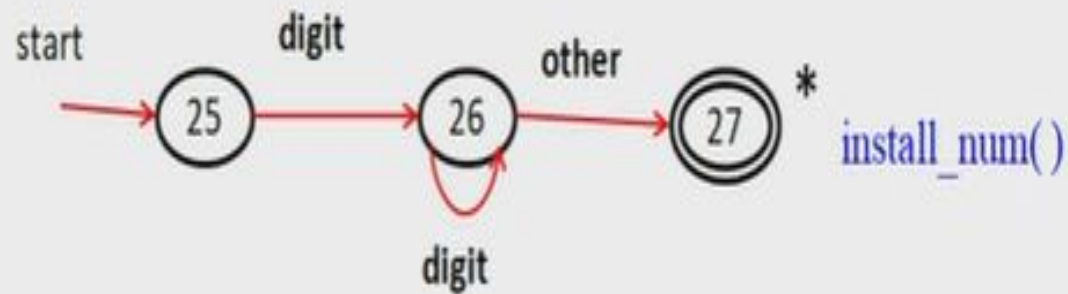
```
if x <= 10
then
    x
else
    y
```

Source program

Lexical
Analyzer

Token	Attribute
IF	NULL
ID	Ptr to x in ST
RELOP	LE
NUM	Ptr to 10 in ST
THEN	NULL
ID	Ptr to x in ST
ELSE	NULL
ID	Ptr to y in ST

Transition Diagram for Unsigned Integer Numeric Constant

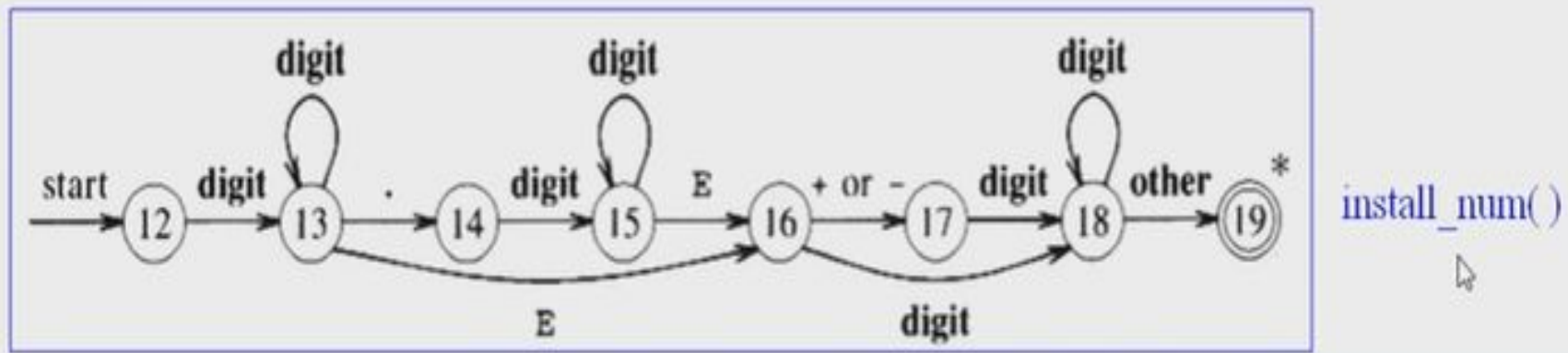


- It accepts the unsigned integer numeric constants
- Example- 314, 9, 41267

Recognition of Tokens

Recognition of Unsigned numbers

Transition Diagram for Unsigned Numbers

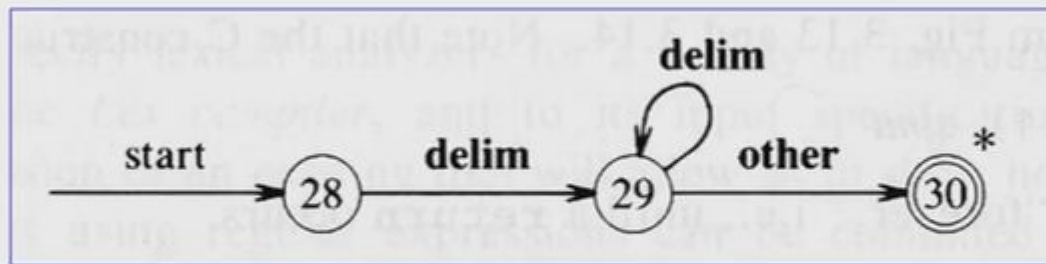


e.g. 12.3E4, 10E15

Recognition of Tokens

Recognition of white space

Transition Diagram for whitespace



- It accepts the whitespaces (newline, tab space and space)
- Returns the token as `ws`

